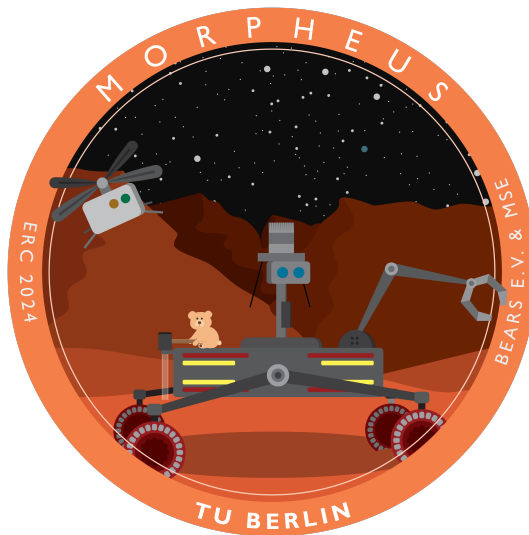


Technical University Berlin

Master of Space Engineering

Navigation (Droning)

PESR2 - MORPHEUS



Author, Affiliation & Matriculation Number:

Vasanthan Kanesh Muruganantham ¹
Technische Universität Berlin
490111

Vyshnav Davanagere Nagaraj ²
Manipal Institute of Technology
210911332

Nithyashree Karunakaramurthy ³
IU International University of Applied Sciences
4243556

October 6, 2024

Advisors:

Simon Stapperfend M. Sc.
Maximilian von Unwerth M. Sc.
Chair of Space Technology
Institute of Aeronautics and Astronautics

Declaration of independence § 60 Abs. 8 AllgStuPO

We hereby declare that we have completed this document independently and single-handedly, without unauthorised assistance and exclusively using the sources and aids listed. We hereby declare in lieu of oath that we have prepared this document independently and by ourselves.

06.10.2024, Berlin

Date, Location



Signature (Droning sub-task Lead)

Contents

List of Figures	V
List of Tables	VI
1 Introduction	1
1.1 Problem Statement	2
1.2 Objectives	3
1.3 Significance	3
2 Concept and Design	4
2.1 Aruco Marker Detection and Tracking	5
2.2 Energy Consumption and Flight Time Analysis	5
3 Assembly, Testing, and Validation	6
3.1 Testing and Validation	9
3.1.1 Software-in-the-Loop (SITL)	9
3.1.2 System Overview of Aruco marker detection	10
3.1.3 Heatmap Generation:	11
3.1.4 Mathematical Foundation	11
3.1.5 Aruco marker detection logic	11
3.1.6 MATLAB analysis	12
3.1.6.1 Data Handling	12
3.1.6.2 Energy and Power Calculations	12
3.1.6.3 Estimation and Output	13
4 Results	14
4.0.1 MATLAB output	14
5 Lessons-Learned	16
Bibliography	i

List of Figures

1.1	MAVEX isometric view 1	1
1.2	MAVEX isometric view 2	1
1.3	Mission Layout	2
3.1	System integration architecture of the MAVEX drone	7
3.2	Wiring Diagram of MAVEX drone	8
3.3	Fully assembled MAVEX top view	9
3.4	Autonomous mission planning using Arducopter SITL [6]	10
3.5	System block diagram of Aruco Marker Detection	11
3.6	Point A	12
3.7	Point C	12
4.1	Flight time estimation during takeoff and landing at checkpoints (B, D)	14
4.2	Best Performance Award - Navigation (Droning)	15

List of Tables

2.1 Components and Specifications of the MAVEX Drone 5

1. Introduction

Project MAVEX (Martial Aerial Vehicle for Exploration) aims to develop a fully autonomous drone engineered for operation within confined environments. Leveraging a combination of GNSS (Global Navigation Satellite System) for external positioning and INS (Inertial Navigation System) for internal navigation, the drone ensures precise and reliable flight, even in challenging spatial conditions. The integration of GNSS and INS enables the drone to traverse predefined waypoints autonomously.[10] At the same time, its capability to execute multiple takeoff and landing sequences within a single mission underscores its suitability to operate in spaces typically restrictive for flexible flight. [15]



Figure 1.1: MAVEX isometric view 1



Figure 1.2: MAVEX isometric view 2

To meet the project's requirements, the drone must maintain a compact footprint of 15 x 15 centimeters, allowing for precise landings and takeoffs on small surfaces. Furthermore, the drone is required to operate effectively within a confined space, specifically a 10-meter cubical enclosure. To achieve these objectives, a high degree of optimization is required in the selection and configuration of key avionics components, including a compact flight controller, electronic speed controllers, and motors, all of which must be precisely integrated without compromising performance.[14]

1.1 Problem Statement

The mission requires the drone to initiate from a designated starting location, autonomously navigate to point A, and capture an image at the specified waypoint. Subsequently, the drone must proceed to point B, execute an emergency landing, and thereafter autonomously take off from point B. The mission continues as the drone flies to point C, where it is tasked with capturing a second image, before returning to the original starting point for its final landing. Throughout the course of this mission, the drone is expected to autonomously perform two takeoffs and two landings, while precisely navigating and completing its assigned tasks without any manual intervention. The successful execution of these autonomous operations within a single mission is critical to the overall objectives of the project.

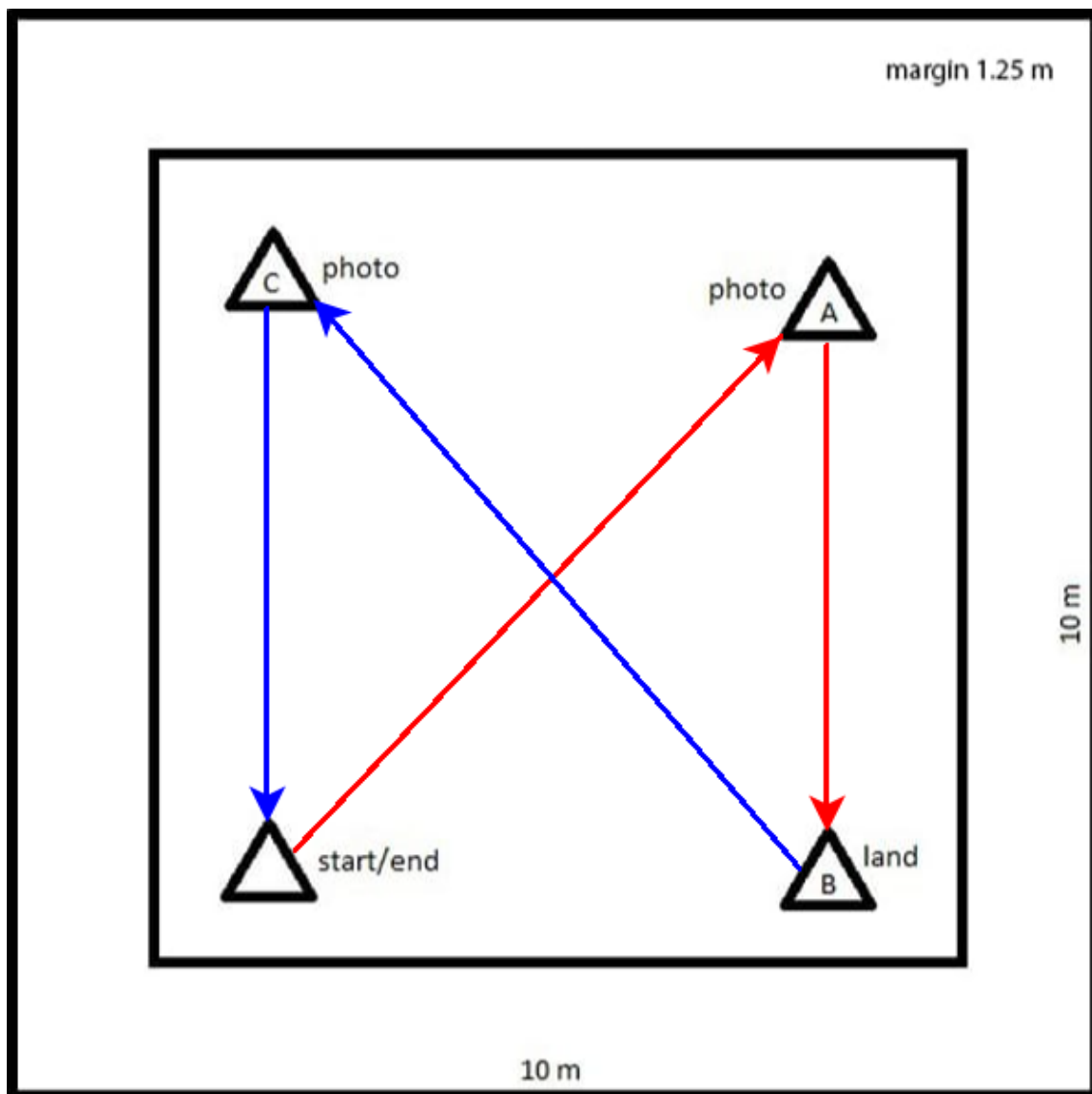


Figure 1.3: Mission Layout

1.2 Objectives

1. To ensure the drone executes fully autonomous operations within a confined area measuring 10 meters by 10 meters.
2. To implement a suite of autonomous functionalities, including automatic takeoff, landing, navigation, delay, arming, and disarming, thereby enabling complete mission automation.
3. To minimize GNSS accuracy errors through real-time error estimation and correction, enhancing the precision of waypoint navigation.
4. To achieve highly precise and reliable autonomous flight by incorporating GPS correction mechanisms to facilitate accurate movement within the constrained environment.

1.3 Significance

The MAVEX drone is equipped with a dual GPS system, which significantly enhances its positional accuracy within confined environments. One GPS unit is responsible for providing a standard three-dimensional fix, while the other delivers a three-dimensional differential fix (DGPS), thereby ensuring superior precision in position holding, even in conditions characterized by substantial magnetic variance or weak GPS signal strength[1]. Furthermore, the drone integrates advanced sensors and utilizes state-of-the-art space-grade wiring, including shielded signal cables designed to mitigate external interference. These features collectively contribute to the drone's robustness and resilience, enabling effective operation in low-GPS environments and ensuring reliable navigation and performance under challenging conditions.

2. Concept and Design

The MAVEX drone is designed with an inverted rotor configuration to maximize thrust while minimizing thrust losses, constructed around a 225mm frame fabricated from fully carbon fiber using CNC machining techniques. This lightweight yet durable structure is equipped with high-grade avionics that support its autonomous operational capabilities. Central to the drone's control system is an advanced F7 processor integrated within the Pixhawk Cube autopilot, which provides substantial processing power through multiple serial ports and UARTs, facilitating the integration of various peripherals and sensors for enhanced functionality.

The propulsion system comprises four units of 1800 Kv motors, delivering a combined thrust of approximately 6 kilograms while maintaining a gross weight of only 960 grams. Coupled with a 4500 mAh battery, the drone offers sufficient endurance for extended missions. Additionally, it features a dual redundant GPS system, with one unit providing a three-dimensional fix and the other offering a three-dimensional differential fix, thereby ensuring precise positioning in environments where GPS signals may be compromised.

Complementary components include a rangefinder, altimeter, and a robust telemetry system designed to facilitate communication and control. The drone's design emphasizes resilience against external interference; analog signal cables and sensitive components are fully shielded from magnetic disturbances through the use of ESD (Electrostatic Discharge) protectors. This shielding ensures optimal functionality of the avionics, even in environments susceptible to electromagnetic interference, thereby rendering the MAVEX highly reliable and effective for autonomous missions within confined spaces. Furthermore, the drone communicates with the ground station via ISM frequencies of 2.4 GHz and 5.8 GHz at a power output of less than 25 mW, in compliance with EU Amateur Radio Regulations. The Comprehensive overview of components incorporated in the MAVEX Drone is given below.[8]

Key Components	Specification
Structure	Aerospace grade carbon fiber frame with 225 mm wheelbase
Propellers	Tri-blade (5153) high-pitch polycarbonate propellers
Motors	1800 Kv BLDC motors with (28mm * 07mm) stator size
ESC	32 Bit 60A 4in1 Electronic Speed Controller
Power module	Power brick mini 15W output
Flight Controller	Pixhawk Cube Black with chibiOS & Ardupilot Frimware
Telemetry	RF Design EU868Ux - (500mW / 868 MHz)
Telecommand	ELRS RP1 - (100mW / 2.4 GHz) Full Duplex Receiver
Video Transmitter	Analog 48CH - (2.5W / 5.8 GHz) VTx
TT&C Antennas	1x Omnidirectional (5.8 GHz) & 2x Dipole (868 MHz)
Primary Camera	Analog 1500TVL 6ms Low Latency CMOS Camera.
Secondary Camera	Digital 4K 60FPS GoPro for HD recording
GPS 1	Radiolink S100 - (1.3GHz, L1 band)
GPS 2	Here 3+ CAN - (1.3GHz, L2 band)
Battery	Tesla Lithium-Ion 6SIP (22.2V / 4.5A / 10C)

Table 2.1: Components and Specifications of the MAVEX Drone

2.1 Aruco Marker Detection and Tracking

The ArUco marker detection system[9] was developed for the European Rover Challenge (ERC) 2024 to detect ArUco markers in real-time, estimate the distance from the drone, and capture images when conditions were met. It ensured compliance with the mission requirements, including capturing images with markers covering at least 5% of the frame with adequate sharpness.

2.2 Energy Consumption and Flight Time Analysis

The flight time analysis[2] shall be conducted using MATLAB. The analysis will utilize the input file 'output.csv', which contains time-stamped data pertaining to battery consumption during flight. MATLAB will process this data, and the results of the analysis will be documented in `flight_time_analysis.xlsx`. Furthermore, MATLAB will generate a graphical representation to visualize the analysis outcomes.

3. Assembly, Testing, and Validation

The development of the MAVEX drone is initiated with the acquisition of high-quality components, followed by the meticulous assembly of its critical parts. This process begins with the integration of motors into a carbon fiber frame, linking the motors to electronic speed controllers (ESCs), which are subsequently connected to the flight controller. Upon completing the assembly, the hardware undergoes rigorous testing to verify proper functionality, before advancing to the software configuration phase. In this stage, the autopilot system is equipped with supplementary components such as dual GPS, a rangefinder, and various sensors, all interfaced through multiple ports. The drone operates in the Ardupilot[3] platform using the Arducopter firmware, configured in Mission Planner to ensure synchronization across navigation, telemetry, and control systems. Figure 3.1 illustrates the sample system architecture of various components, reflecting the connections implemented in the MAVEX drone.

A pivotal aspect of the development process is the tuning of the drone. Thanks to its robust avionics integration, the drone is capable of auto-tuning[11], enabling it to autonomously learn and optimize key flight parameters such as angular rate and momentum. This self-calibration significantly enhances the drone's stability and overall performance. Following the auto-tuning phase, semi-autonomous flight tests are conducted, including a position hold test to verify the drone's ability to maintain a stable hover, and a Return-to-Home (RTH) test to evaluate its precision in navigating back to the original takeoff point.

Upon successful completion of these tests, the drone proceeds to execute its primary mission: achieving fully autonomous flight[16] with a single takeoff and landing. Throughout this mission, flight logs are meticulously analyzed to assess factors such as flight stability, path-following accuracy, and resistance to electromagnetic interference. This comprehensive testing process ensures that the MAVEX drone is fully prepared for safe and reliable operation in subsequent missions, demonstrating its readiness for real-world applications.

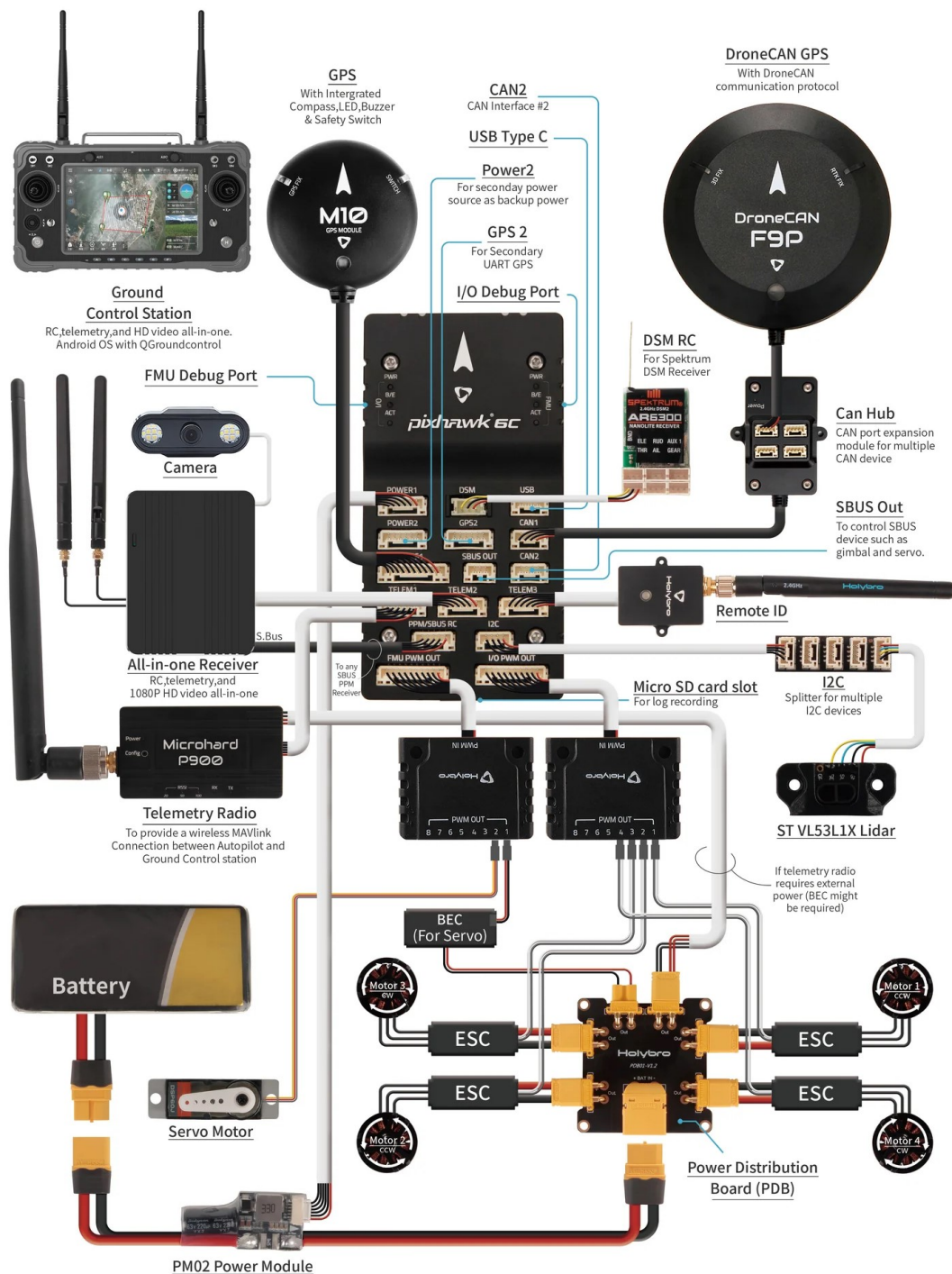


Figure 3.1: System integration architecture of the MAVEX drone

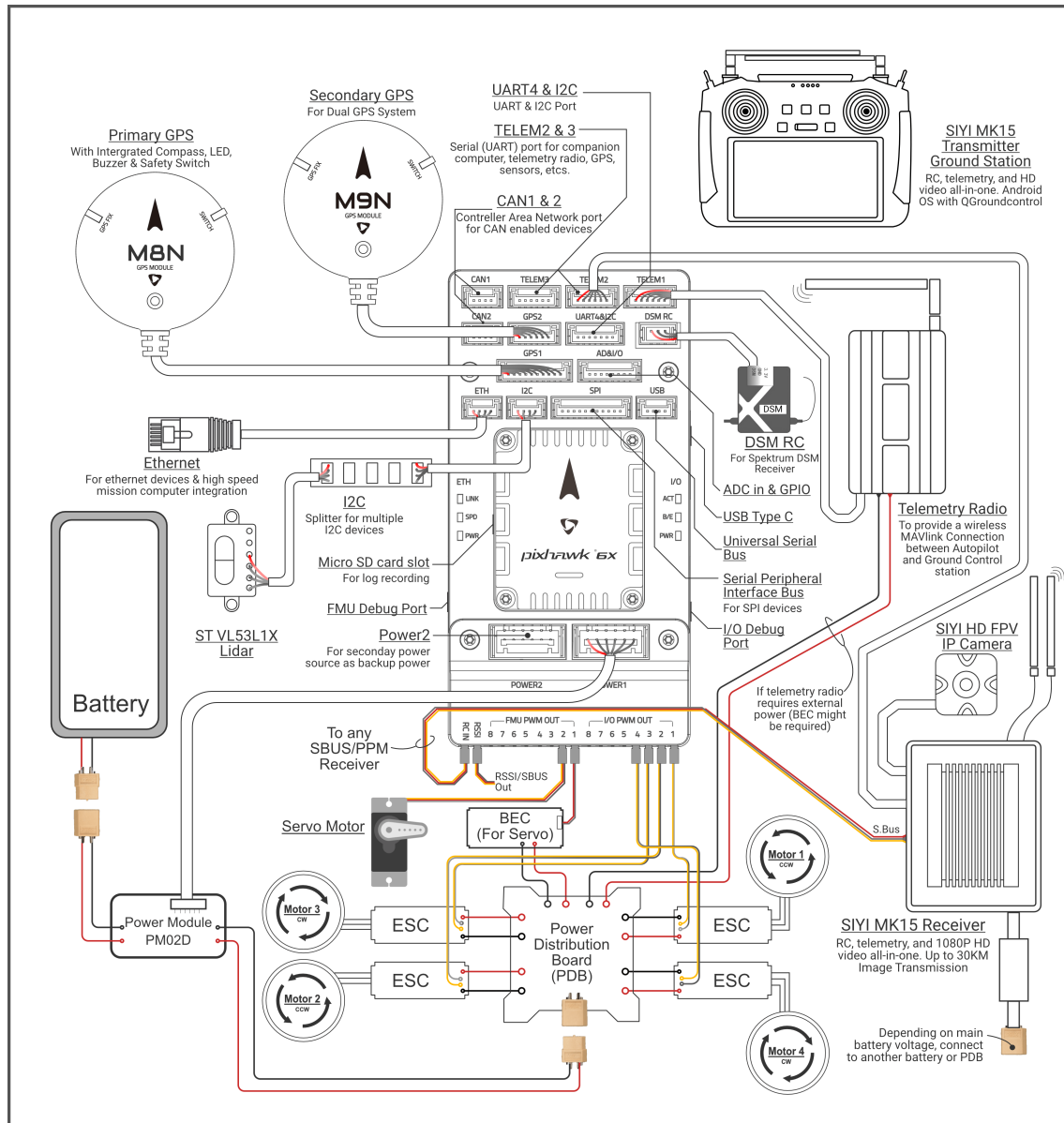


Figure 3.2: Wiring Diagram of MAVEX drone



Figure 3.3: Fully assembled MAVEX top view

3.1 Testing and Validation

The testing and validation process for the MAVEX drone was conducted through a multifaceted approach, utilizing three key methods: Software-in-the-Loop (SITL), automatic detection of Aruco markers via Python, and MATLAB/Simulink for endurance and energy estimation.

3.1.1 Software-in-the-Loop (SITL)

The SITL[4] method provided a comprehensive testing platform using the Mission Planner's integrated simulator. This allowed for the execution of the fully autonomous mission in a virtual environment. Configured with the Arducopter framework, the simulation was designed to replicate the drone's intended real-world flight path by assigning waypoints. The primary objective was to validate the drone's capability to perform multiple takeoffs and landings within a single mission.

During the SITL simulation, the mission profile was structured as follows: the drone initiated takeoff from a designated starting point and navigated to point A, where it autonomously captured an image. The drone then proceeded to point B, performed a controlled descent, disarmed, and remained stationary for a few seconds before re-arming. Upon re-arming, the drone took off again and traveled to point C to capture another image. Finally, the drone returned to the starting point (point D),

where it executed an automatic descent, landing, and disarming sequence to complete the mission.

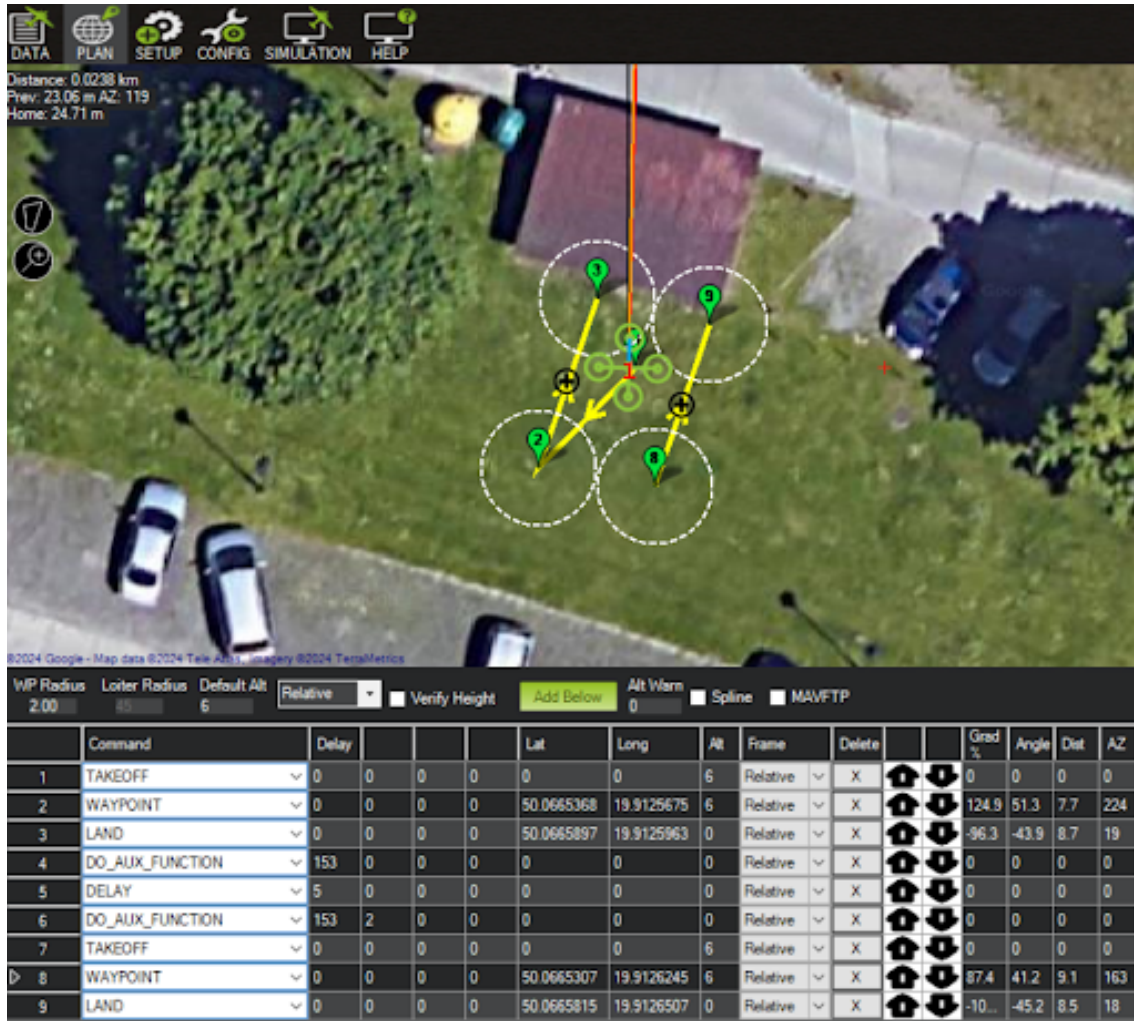


Figure 3.4: Autonomous mission planning using Arducopter SITL [6]

This simulation successfully validated the drone's capability to autonomously perform all mission-critical tasks, including multiple takeoffs, landings, and image captures, thus confirming the reliability of the MAVEX system under test conditions.

The subsequent step involves integrating the drone's autonomous capabilities with real-time image sensing for Aruco marker detection, utilizing Python scripts executed through on-ground computation via live, low-latency video transmission.

3.1.2 System Overview of Aruco marker detection

Camera Setup: The camera feed is initialized using OpenCV's [5] VideoCapture, enabling real-time detection.

Marker Detection: The detection utilizes OpenCV's DICT_ARUCO_ORIGINAL, with DetectorParameters[4]. Detected markers are highlighted with green polylines and labeled with their IDs.

Distance Estimation: The distance is calculated using:

$$\text{Distance} = \frac{\text{Focal Length} \times \text{Real Marker Size}}{\text{Pixel Size}} \quad (3.1)$$

3.1.3 Heatmap Generation:

A heatmap is created based on proximity using an exponential decay function:

$$\text{Heat Intensity} = \min(255, 255 \times e^{-d}) \quad (3.2)$$

where d is the distance in meters.

Image Capture: Images are captured whenever a new marker is detected, or when a previously detected marker is seen again after 2 seconds.

Real-Time Feedback: Three outputs are updated in real-time: the original feed with highlighted markers, a binary image showing the markers, and a heatmap of proximity.

3.1.4 Mathematical Foundation

The system uses geometric relationships for distance estimation, based on the known marker size and focal length. An exponential decay function[13] models proximity for heatmap generation, offering immediate feedback on the drone's position relative to the markers.

3.1.5 Aruco marker detection logic

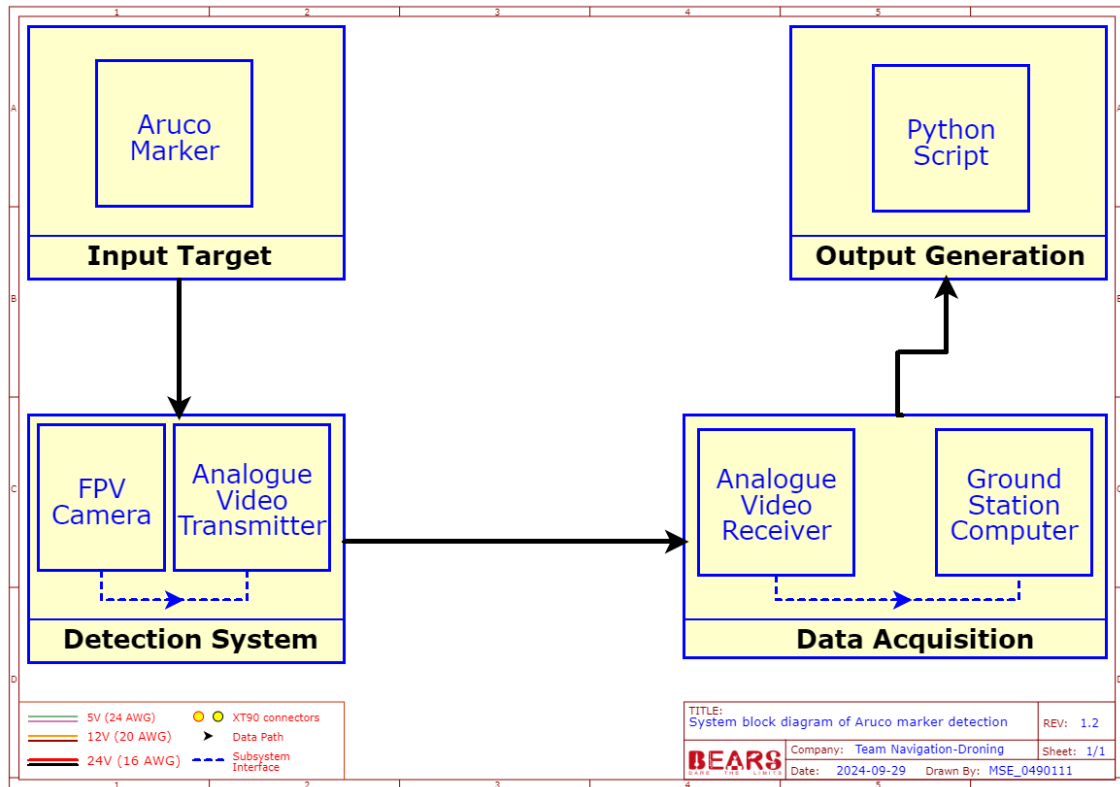


Figure 3.5: System block diagram of Aruco Marker Detection

This system integrates computer vision, geometric distance estimation, and heatmap visualization to ensure accurate detection and image capture of ArUco markers, meeting the ERC 2024 mission requirements with real-time feedback and robust performance.

In reference to Figure 3.5, the Aruco marker detection system is integrated with the drone's autonomous mission, utilizing a Python program running in the background to identify and capture Aruco markers from the video feed. The video feed is transmitted via an analog camera and transmitter, operating at a resolution of 640 x 480 pixels with a low latency of less than 4 milliseconds. While the camera ensures fast and high-quality imagery, the processing of the Aruco markers occurs at the ground station rather than onboard the drone. As the drone passes over points A and C, the Python program promptly detects the Aruco markers within the video feed, captures the corresponding images, and stores them in a predesignated file. This process facilitates an automated and efficient photo-capture functionality, independent of the primary mission planner configuration, ensuring accurate and reliable marker detection and data acquisition throughout the mission.



Figure 3.6: Point A

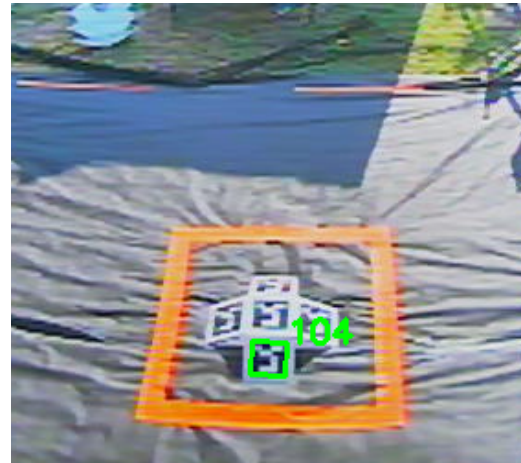


Figure 3.7: Point C

3.1.6 MATLAB analysis

The MATLAB code employs the `detectImportOptions` function to configure the import settings for the CSV file (`output.csv`), particularly specifying the datetime format for the `Time_Stamp` column. Data is imported into a table, and the relevant timestamps, voltage, and current values are extracted for analysis.

3.1.6.1 Data Handling

The timestamps are converted to datetime objects to facilitate calculations of time intervals. Any missing values in the voltage and current columns are initially assigned a value of zero and then linearly interpolated to ensure data continuity.

3.1.6.2 Energy and Power Calculations

Power consumption, measured in watts, is calculated by multiplying the voltage and current values. Subsequently, cumulative energy consumption and remaining energy

are computed over time. The remaining flight time is estimated by analyzing the average power consumption and the available energy.

3.1.6.3 Estimation and Output

The energy metrics are estimated, and a comprehensive report is generated in an Excel file. The code calculates the average power consumption and the total available energy, considering a 20% buffer for safety. The final results are compiled into an Excel report `flight_time_analysis.xlsx`, which includes the following columns: *Timestamp*, *Voltage (V)*, *Current (A)*, *Power Consumption (W)*, *Energy Consumed (J)*, *Average Power Consumption (J/s)*, *Remaining Energy (kJ)*, and *Estimated Remaining Flight Time*.

MATLAB generates informative visualization in figure 3.8 illustrating energy consumption over time, which are particularly effective in highlighting critical phases of flight, such as takeoff and landing. Key visualizations include:

Energy Consumption vs. Time: This graph illustrates the variation in energy consumption throughout the flight. By plotting energy consumption against flight time, operators can identify periods of elevated power usage, which typically correspond to the takeoff, ascent, and landing phases. This information can guide strategies for optimizing flight paths and managing battery usage.

Remaining Energy Over Time: A second plot presents the remaining battery energy throughout the flight[12]. This visualization enables operators to assess the amount of energy available at various intervals, facilitating real-time decision-making regarding the continuation of the mission or the initiation of a safe landing.

Power Consumption Analysis: Furthermore, power consumption[7] can be plotted over time to illustrate fluctuations associated with the drone's activities, such as acceleration, cruising, and descent. This analysis provides additional context for understanding energy usage patterns during different stages of flight.

4. Results

The outcomes of the MAVEX drone project demonstrate notable achievements during the various testing phases. The software-in-the-loop (SITL) simulations proved to be highly effective, facilitating the successful execution of missions that involved multiple takeoffs and landings. The autonomous navigation capabilities of the drone performed exceptionally well, with the Aruco marker detection system operated by a Python program functioning with both speed and accuracy. Notably, despite the presence of metallic structures that induced magnetic interference and limited GPS penetration within the confined test environment, the drone's real-time performance aligned with the expectations established by the simulations.

The integration of MATLAB simulations was key in optimising the drone's energy consumption and telemetry accuracy. By comparing actual power consumption with the ideal data, MATLAB helped ensure the drone could efficiently manage its power, avoiding false triggers of the failsafe mode that might otherwise interrupt the mission prematurely. This feature allowed for maximum utilisation of the drone's energy resources during the flight.

4.0.1 MATLAB output

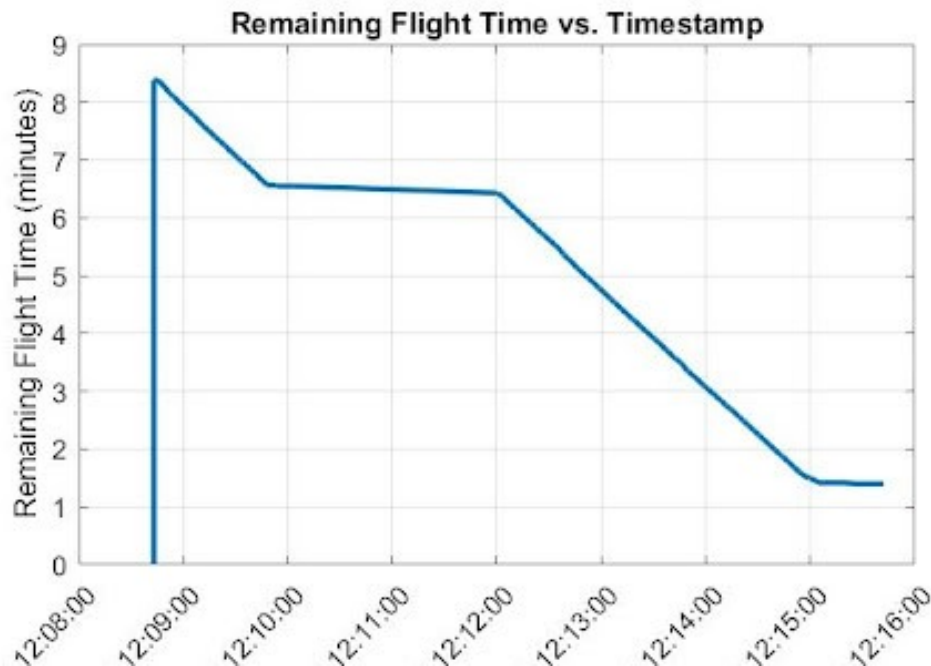


Figure 4.1: Flight time estimation during takeoff and landing at checkpoints (B, D)

As indicated by the graph in Figure 4.1, the analysis of real-time telemetry data yields the following results:

- **Estimated Energy Consumption Rate:** Average power consumption in watts
- **Estimated Total Energy Available:** Total energy in kilojoules, including a 20% buffer.
- **Estimated Remaining Flight Time:** Remaining flight time formatted in minutes and seconds.

In summary, the MAVEX drone effectively demonstrated the capability to execute fully autonomous missions within a confined environment. Through meticulous design and the integration of high-quality components, including dual GPS systems for enhanced accuracy, a robust autopilot system, and advanced sensor technology, the drone successfully navigated multiple checkpoints. The software-in-the-loop simulations, supported by Python programming and MATLAB analysis, contributed to optimal performance, particularly in terms of energy management and accuracy during real-time operations. Despite the challenges posed by magnetic interference and limited GPS penetration, the drone's navigation capabilities underscored its stability and reliability, meeting all primary objectives of the project.

As a result, the MAVEX drone earned a total score of 224 out of 300 points, securing the **Best Performance Award in the Navigation (Droning) sub-task** at the world finals of the European Rover Challenge (ERC) 2024.



Figure 4.2: Best Performance Award - Navigation (Droning)

5. Lessons-Learned

Throughout the development and execution of the Mavex project, several key lessons emerged, both on the technical and operational fronts. The experience proved invaluable in advancing my understanding of autonomous systems, interdisciplinary collaboration, and the challenges of working within complex environments.

1. Navigating Complex Environments: One of the most significant lessons learned was the challenge of operating an autonomous drone in confined spaces with limited GNSS signals. The project emphasized the necessity of precise navigation systems, particularly in environments with substantial electromagnetic interference, such as those with metallic structures. To overcome these limitations, the implementation of a differential GPS system proved instrumental, providing a level of accuracy that allowed for smoother waypoint navigation. The process taught me the importance of adapting to environmental constraints and developing solutions tailored to specific operational contexts.

2. Real-time Data Processing: The task of real-time ArUco Marker detection highlighted the complexities involved in processing and interpreting data instantaneously. Using both Python and MATLAB, it became evident that code optimization and algorithm efficiency were critical to maintaining the speed and accuracy required for successful marker detection. The lesson here extended beyond the technical implementation; it reinforced the value of iterative testing and continuous improvement in ensuring that the system could meet real-world demands. Ensuring accuracy while balancing processing speed was a challenging but rewarding aspect of the project.

3. Hardware-Software Integration: Integrating hardware components such as cameras and sensors with the software architecture presented another key learning point. Achieving a seamless interaction between the drone's hardware and the Python/MATLAB code was essential for reliable autonomous operation. This project reinforced the importance of understanding both hardware limitations and software capabilities, and how the two must work harmoniously to create a functioning autonomous system. The synchronization of real-time data inputs from various sensors with software output underscored the need for meticulous planning and testing in system design.

4. Autonomous Systems and Iterative Testing: The development of a fully autonomous system was both a technical challenge and an opportunity to learn about the iterative nature of testing. Early-stage tests revealed unexpected behaviors, particularly in the drone's ability to consistently detect ArUco Markers and execute accurate waypoint navigation. Regular, iterative testing allowed for gradual refinement of both hardware performance and software algorithms, ensuring reliability

across multiple test runs. This iterative process taught me the importance of patience and persistence in system development, as well as the necessity of robust testing procedures to identify and address potential points of failure.

5. Interdisciplinary Collaboration: Working alongside a team with expertise in Python programming and MATLAB simulations provided a valuable lesson in the importance of collaboration across disciplines. Clear communication between team members, especially when integrating different components of the project, was critical for success. The project highlighted the need for a common understanding of goals and methodologies, ensuring that all contributors were aligned in their efforts to achieve a unified outcome. This collaboration also reinforced the value of teamwork in solving complex problems, where diverse skill sets can contribute to a more comprehensive and effective solution.

6. Adaptability and Problem Solving: As with any complex project, unexpected challenges arose that required adaptability and quick problem-solving. Technical constraints, such as signal interference or system limitations, demanded improvisation and on-the-spot decision-making. Through these challenges, I learned the importance of flexibility in project management, particularly when working within a tight timeline. This experience honed my ability to assess problems rapidly, propose feasible solutions, and implement changes efficiently, ensuring that the project remained on track.

7. System Limitations and Future Improvements: The project also provided insights into the limitations of the current system. For instance, real-time data processing sometimes led to minor delays, which impacted the overall performance during certain test runs. Additionally, while the differential GPS system improved navigation, the quality of signal reception in particularly dense environments remained an area for improvement. Identifying these limitations was an important step toward refining the system for future iterations and highlighted areas for further research and development, such as enhancing the drone's stability in high-interference environments or exploring alternative sensors for navigation in GNSS and GPS-denied spaces.

8. Personal Growth and Achievement: Finally, this project represented a significant personal achievement, culminating in the successful navigation and ArUco Marker detection that contributed to winning the best performance award in the drone navigation category. Beyond the technical accomplishments, the project reinforced my capabilities as a leader and team member, deepening my confidence in managing complex systems and interdisciplinary collaborations. The lessons learned throughout the project have undoubtedly shaped my approach to future endeavors in autonomous systems development and further solidified my passion for innovative solutions in the field of aerial robotics.

Bibliography

- [1] Fadi Al-Turjman. A novel approach for drones positioning in mission critical applications. *Transactions on Emerging Telecommunications Technologies*, 33(3):e3603, 2022. (cited on Page 3)
- [2] Marcin Biczyski, Rabia Sehab, James F Whidborne, Guillaume Krebs, and Patrick Luk. Multirotor sizing methodology with flight time estimation. *Journal of Advanced Transportation*, 2020(1):9689604, 2020. (cited on Page 5)
- [3] He Bin and Amahah Justice. The design of an unmanned aerial vehicle based on the ardupilot. *Indian Journal of Science and Technology*, 2(4):12–15, 2009. (cited on Page 6)
- [4] Adriano Bittar, Helosman V Figueredo, Poliana Avelar Guimaraes, and Alessandro Correa Mendes. Guidance software-in-the-loop simulation using x-plane and simulink for uavs. In *2014 International Conference on Unmanned Aircraft Systems (ICUAS)*, pages 993–1002. IEEE, 2014. (cited on Page 9 und 10)
- [5] Ivan Culjak, David Abram, Tomislav Pribanic, Hrvoje Dzapov, and Mario Cifrek. A brief introduction to opencv. In *2012 proceedings of the 35th international convention MIPRO*, pages 1725–1730. IEEE, 2012. (cited on Page 10)
- [6] Aicha Idriss Hentati, Lobna Krichen, Mohamed Fourati, and Lamia Chaari Fourati. Simulation tools, environments and frameworks for uav systems performance analysis. In *2018 14th International Wireless Communications & Mobile Computing Conference (IWCMC)*, pages 1495–1500. IEEE, 2018. (cited on Page V und 10)
- [7] Dooyoung Hong, Seonhoon Lee, Young Hoo Cho, Donkyu Baek, Jaemin Kim, and Naehyuck Chang. Least-energy path planning with building accurate power consumption model of rotary unmanned aerial vehicle. *IEEE Transactions on Vehicular Technology*, 69(12):14803–14817, 2020. (cited on Page 13)
- [8] Adam Juniper. *The Complete Guide to Drones Extended 2nd Edition*. Hachette UK, 2015. (cited on Page 4)
- [9] Igor Lebedev, Aleksei Erashov, and Aleksandra Shabanova. Accurate autonomous uav landing using vision-based detection of aruco-marker. In *International Conference on Interactive Collaborative Robotics*, pages 179–188. Springer, 2020. (cited on Page 5)

-
- [10] Thomas Lee, Susan Mckeever, and Jane Courtney. Flying free: A research overview of deep learning in drone navigation autonomy. *Drones*, 5(2):52, 2021. (cited on Page 1)
 - [11] Antonio Loquercio, Alessandro Saviolo, and Davide Scaramuzza. Autotune: Controller tuning for high-speed flight. *IEEE Robotics and Automation Letters*, 7(2):4432–4439, 2022. (cited on Page 6)
 - [12] Pedro Alexandre Lourenço Moutinho. Real-time estimation of remaining uav flight time based on the total energy balance. Master’s thesis, Universidade da Beira Interior (Portugal), 2017. (cited on Page 13)
 - [13] SW Provencher. A fourier method for the analysis of exponential decay curves. *Biophysical journal*, 16(1):27–41, 1976. (cited on Page 11)
 - [14] Quan Quan. *Introduction to multicopter design and control*. Springer, 2017. (cited on Page 1)
 - [15] Virginia Santamarina-Campos and Marival Segarra-Oña. Introduction to drones and technology applied to the creative industry. airt project: An overview of the main results and actions. *Drones and the Creative Industry: Innovative Strategies for European SMEs*, pages 1–17, 2018. (cited on Page 1)
 - [16] G Anup Venkatesh, P Sumanth, and KR Jansi. Fully autonomous uav. In *2017 International Conference on Technical Advancements in Computers and Communications (ICTACC)*, pages 41–44. IEEE, 2017. (cited on Page 6)